

ENTERPRISE THREAT INTELLIGENCE WHITEPAPER

NO ENTRY (SOC-AI)

Complete Architecture, Data Pipeline, Function Breakdown, and
Implementation Engineering Reference

PLATFORM:

NO ENTRY Cybersecurity Intelligence Platform

PRODUCTION URL:

<https://soc-ai-six.vercel.app/>

REPOSITORY:

ceylonroameryt-bit/SOC-AI

BRANCH:

fix/feed-reliability-and-health

DATE:

September 26, 2026

DOCUMENT CLASSIFICATION:

Technical Architecture Specification (Public /
Open Source)

1. Executive Summary & Non-Technical Overview

1.1 What is NO ENTRY?

NO ENTRY (also engineered as **SOC-AI**) is a next-generation cybersecurity threat intelligence platform that automates the collection, validation, de-hyping, enrichment, and MITRE ATT&CK® mapping of real-world cybersecurity intelligence. It transforms over 1,000 disparate vendor research blogs, government CSIRT feeds, ransomware leak trackers, and vulnerability repositories into actionable operational telemetry for Security Operations Centers (SOCs).

1.2 The Problem It Solves

- **Severe Information Overload & Alert Fatigue:** Security analysts are overwhelmed by hundreds of disconnected intelligence feeds published across the internet every hour.
- **Vendor Marketing Noise & "FUD":** A vast portion of public cybersecurity content consists of sales webinars, product pitches, venture capital rounds, and opinion columns disguised as urgent zero-day alerts.
- **Manual Framework Alignment:** Security teams spend hours manually cross-referencing articles with adversary Tactics, Techniques, and Procedures (TTPs) defined in the MITRE ATT&CK framework.
- **Uncorrelated Vulnerabilities:** Analysts lack immediate, evidence-grounded visibility into whether a referenced CVE is actively weaponized and listed on the CISA Known Exploited Vulnerabilities (KEV) catalog.
- **Legal Copyright Exposure:** Scraping full articles without explicit licensing or fair-use snippet limits exposes organizations to copyright liability.

1.3 Core Business & SOC Value Proposition

Dimension	Traditional Manual SOC Operation	NO ENTRY Automated Platform
Intelligence Gathering	Manual browsing of 50+ websites & Twitter/X feeds	Automated ingestion from 1,000+ monitored endpoints
Content Quality	High noise; webinars & marketing trigger false alarms	Multi-factor severity engine dampens marketing by -50 pts
Framework Tagging	Manual, ad-hoc tagging of MITRE techniques	Automated heuristic mapping across 14 Tactics & 52 Techniques
CVE Prioritization	Manual cross-checking against NVD and KEV	Real-time CISA KEV bonus (+35 pts) & automated IOC extraction
Legal Compliance	Uncontrolled full-text scraping risks copyright suits	Strict fair-use 280-500 char snippet limits & mandatory backlinks

2. End-to-End System Architecture & Data Flow

2.1 High-Level Architecture Diagram



2.2 Data Ingestion Lifecycle Sequence

1. **Scheduler Invocation:** Background timer or manual API request initiates `server/collector.js`.

2. **Run Initialization:** `startCollectionRun()` assigns a unique `runId` (e.g. `run-1790454547760-d899f6`) and logs start timestamp to `collection_runs.json`.
3. **Security Verification:** Feeds are validated via `isPrivateOrInternalUrl()` to ensure no SSRF requests target internal resources.
4. **Conditional Fetching:** HTTP requests include `If-None-Match` and `If-Modified-Since`. Feeds returning HTTP 304 are counted as verified without consuming download bandwidth.
5. **Payload Ingestion:** XML/Atom feeds are parsed with a 32MB payload ceiling, safely handling large bulletins such as CISA US-CERT archives.
6. **Deterministic Processing:** Article IDs are derived using SHA-256 hashes of canonical links. Publication dates are parsed strictly without current-time substitution.
7. **Severity & MITRE Scoring:** Multi-factor scoring evaluates CISA KEV presence, active exploitation phrases, and marketing dampeners while tagging MITRE techniques.
8. **Atomic Persistence:** Processed articles are committed to `server/data/news.json` and PostgreSQL. Cache invalidation ensures instant visibility on API routes.
9. **Run Completion:** `completeCollectionRun()` logs attempted, succeeded, failed, and skipped source counts alongside entry rejection reasons.

3. Technical Module & Function Breakdown

3.1 Ingestion & Network Guard: `server/utils/urlUtils.js`

Protects the host environment against Server-Side Request Forgery (SSRF) and prevents attackers from using the ingestion engine to probe cloud metadata services or internal networks.

```
export function isPrivateOrInternalUrl(rawUrl) {
  try {
    const parsed = new URL(rawUrl);
    const host = parsed.hostname.toLowerCase();

    // Loopback and local domain blocking
    if (host === 'localhost' || host.endsWith('.local') || host.endsWith('.internal')) return true;

    // Cloud Instance Metadata Service (IMDS) protection
    if (host === '169.254.169.254' || host === 'metadata.google.internal') return true;

    // RFC 1918 Private IPv4 address blocks
    if (/^127\./.test(host) || /^10\./.test(host) || /^192\.168\./.test(host)) return true;
    if (/^172\.(1[6-9]|2[0-9]|3[0-1])\./.test(host)) return true;

    return false;
  } catch {
    return true; // Deny unparseable URLs safely
  }
}
```

3.2 Ingestion Engine: `server/services/collectorEngine.js`

Manages HTTP connections, body buffers, and publisher limits:

- `MAX_BODY_BYTES = 32 * 1024 * 1024` (32MB limit ensures massive security bulletins are digested without memory crashes).
- `withHostLock(host, task)` serializes outgoing requests per domain, preventing concurrency bursts.
- `parseRetryAfter(header)` respects publisher rate-limiting policies on HTTP 429.

3.3 Multi-Factor Severity Engine: `server/services/severityEngine.js`

Replaces naive keyword matching with evidence-grounded scoring:

- **CISA KEV Match (+35 pts):** Checked against known active vulnerabilities.
- **Active Exploitation Signals (+30 pts):** Phrases like `"exploited in the wild"`, `"zero-day"`.
- **RCE & Ransomware Signals (+20-25 pts):** Unauthenticated remote code execution, leak sites.

- **Marketing Dampeners (-50 pts):** Terms like "webinar", "fireside chat", "whitepaper", "seed funding", "gartner" downgrade score to Informational or Low.

```
export function assessSeverity(article) {
  const text = `${article.title || ''} ${article.contentSnippet || ''}`.toLowerCase();

  // Immediate dampening for promotional/editorial content
  if (INFORMATIONAL_SIGNALS.some(s => text.includes(s))) {
    return { severity: 'Informational', score: 10, evidenceStatus: 'unassessed' };
  }

  let score = 20;
  if ([...KNOWN_KEY_CVES].some(c => text.includes(c.toLowerCase()))) score += 35;
  if (ACTIVE_EXPLOITATION_SIGNALS.some(s => text.includes(s))) score += 30;

  if (score >= 70) return { severity: 'Critical', score, evidenceStatus: 'verified' };
  if (score >= 50) return { severity: 'High', score, evidenceStatus: 'verified' };
  if (score >= 35) return { severity: 'Medium', score, evidenceStatus: 'advisory' };
  return { severity: 'Low', score, evidenceStatus: 'unassessed' };
}
```

3.4 MITRE ATT&CK Mapping:

server/services/mitreService.js

Classifies reports across 14 enterprise tactics:

Tactic ID	Tactic Name	Key Monitored Techniques	Color Code
TA0043	Reconnaissance	Active Scanning (T1595), Gather Victim Info (T1589)	#38bdf8
TA0001	Initial Access	Phishing (T1566), Exploit Public-Facing App (T1190)	#f87171
TA0002	Execution	Command & Scripting (T1059), Scheduled Task (T1053)	#fb923c
TA0003	Persistence	Boot/Logon Autostart (T1547), Web Shell (T1505)	#facc15
TA0005	Defense Evasion	Obfuscated Files (T1027), Disable Tools (T1562)	#e879f9
TA0040	Impact	Data Encrypted for Impact (T1486), Service Stop (T1489)	#ef4444

3.5 Legal Permissions & Fair-Use:

server/services/permissionService.js

- **Snippet Length Enforced:** Caps article summaries to 280–500 characters, preventing reproduction of copyrighted full-text articles.
- **Image Hotlinking Blocked:** Eliminates asset bandwidth leeching when disallowed by publisher terms.
- **Mandatory Attribution:** Every record preserves canonical backlink requirements (attributionRequired: true).

3.6 Timezone-Aware Explore Engine: `server/routes/explore.js`

Converts local user midnight into exact UTC ISO timestamp boundaries, preventing timezone drift:

```
export function getLocalMidnightUTC(dateStr, timeZone) {
  const [y, m, d] = dateStr.split('-').map(Number);
  let guess = Date.UTC(y, m - 1, d, 0, 0, 0);
  // Iterative convergence using Intl.DateTimeFormat
  for (let iter = 0; iter < 3; iter++) {
    const parts = new Intl.DateTimeFormat('en-US', {
      timeZone, year: 'numeric', month: 'numeric', day: 'numeric',
      hour: 'numeric', minute: 'numeric', second: 'numeric', hour12: false
    }).formatToParts(new Date(guess));
    // Compute drift and adjust guess
  }
  return new Date(guess).toISOString();
}
```

3.7 Vercel Serverless Parity Bridge: `api/index.js`

Eliminates divergent production vs. local behavior by directly routing all incoming serverless requests through the canonical Express application:

```
import app from '../server/server.js';

export default function handler(req, res) {
  return app(req, res);
}
```

4. Database Schema & Storage Persistence

4.1 PostgreSQL Schema (Migration 002)

Located in `server/db/migrations/002_evidence_and_runs_schema.sql`, providing enterprise relational persistence:

- `sources` : Stores configured feeds, expected polling intervals, and provenance.
- `collection_runs` : Records execution telemetry (timestamps, duration, HTTP status, parsed vs. rejected tallies).
- `threat_assessments` : Stores multi-factor severity scores, CISA KEV flags, and priority classifications.
- `analyst_actions` : Audit trail of analyst status changes (New → Investigating → Closed), notes, and dismissal reasons.

4.2 Resilient File-Based JSON Fallback

When PostgreSQL is disconnected, the platform seamlessly operates using atomic JSON document storage:

- `server/data/news.json` : Active news archive with SHA-256 deduplicated records.
- `server/data/sources_registry.json` : 1,055 catalogue registry with review states.
- `server/data/feed_health.json` : 24h rolling operational health and latency measurements.
- `server/data/source_permissions.json` : Complete legal permission matrix.
- `server/data/collection_runs.json` : Immutable historical telemetry runs.

5. Frontend Architecture & Workspaces

5.1 Routing Tree (React 19 + TypeScript + Vite)

```
NO ENTRY SHELL (src/App.tsx)
├─ / (or /intelligence) → IntelligenceWorkspace.tsx (Live threat radar, category filters, anal
├─ /overview             → Dashboard.tsx (Severity charts, KEV tracker, KPI cards)
├─ /explore              → Explore.tsx (Date presets, timezone selector, keyword search)
├─ /sources              → Sources.tsx (1,000 target progress, health telemetry, permissions)
├─ /vulnerabilities      → VulnerabilitiesView.tsx (Active CVEs & CISA KEV table)
├─ /mitre                → MitreHeatmap.tsx (14-tactic interactive ATT&CK matrix)
├─ /mitre-news           → MitreNews.tsx (Technique-filtered report feeds)
├─ /rules                → RuleLibrary.tsx (YARA & Sigma detection rule library)
├─ /reports              → ReportsHub.tsx (Automated daily PDF/CSV/STIX 2.1 report generation)
└─ /settings             → Settings.tsx (Perimeter watchlist assets, webhooks, SIEM API keys)
```

5.2 Key User Workspaces

- **Intelligence Workspace:** Real-time incident triage feed with live status transitions, severity badges, and quick MITRE tags.
- **Explore Archive:** Timezone-aware date search with presets (Today, Yesterday, Last 7d, Last 30d) and a slide-over detail drawer.
- **Sources Hub:** Reconciled 1,000-source target progress bar, operational health scorecard, latency indicators, and permission status.

6. Development & Deployment Guide

6.1 Prerequisites & Tech Stack

- Node.js >= 20.0.0
- npm >= 10.0.0
- Express 5.2 | React 19.2 | TypeScript 5.7 | Vite 7.3

6.2 Development Commands

```
# Install dependencies
npm install

# Start frontend development server (http://localhost:5173)
npm run dev

# Start backend server with Nodemon (http://localhost:3000)
npm run server

# Run manual threat crawl across active feeds
npm run collector

# Execute full automated test suite & production bundle
npm test
```

6.3 Automated Test Suite (98 Tests / 18 Suites)

Verification State: 100% Passing

All 98 automated regression tests pass cleanly via `npm test`, including SSRF protections, conditional requests, stable SHA-256 IDs, timezone midnight math, run-level telemetry, and the Vercel Serverless parity bridge.

6.4 Deployment Instructions

To push changes to the live production deployment at `https://soc-ai-six.vercel.app/`:

```
git push origin fix/feed-reliability-and-health
```

Vercel automatically executes `npm run build` and routes all `/api/*` endpoints through the repaired `api/index.js` bridge, immediately exposing `/api/explore` and live run telemetry in production.